

Automatic Archiving of DynamoDB Data using AWS Lambda To Begin with the Lab

Summary of the Lab

An automatic archival solution was created for expired **Amazon DynamoDB** items using **Time to Live (TTL)**, **DynamoDB Streams**, and **AWS Lambda**. A new archive table was created, and a Lambda function was configured to copy expired records from the source table into the archive table whenever DynamoDB removed them through TTL. The Lambda execution role was updated with the required IAM permissions to perform PutItem operations on the archive table. Finally, TTL was enabled on the source table, the stream trigger was activated, and the solution was verified by confirming that expired records appeared in the archive table and Lambda logs.

Understanding Time to Live (TTL) Archiving in Amazon DynamoDB

Amazon DynamoDB Time to Live (TTL) automatically deletes items after a specified expiration timestamp, helping keep frequently accessed ("hot") data separate from older ("cold") data. By combining TTL with **DynamoDB Streams** and **AWS Lambda**, every expired item can be captured before it disappears permanently and archived into another DynamoDB table or **Amazon S3**. This keeps the primary table small, reduces storage costs, and preserves historical records for future analysis without affecting application performance.

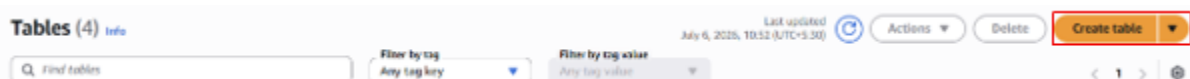
Example:

A CustomerOrders table stores active orders with an expires attribute set to 30 days after delivery. When an order expires, DynamoDB removes it automatically, DynamoDB Streams generates a REMOVE event, and AWS Lambda copies the old record into an CustomerOrdersArchive table. The active table remains optimized for current orders while historical data stays available for reporting.

Lab Steps

Step 1: Create table

- Sign in to the **AWS Management Console**.
- Navigate to **Amazon DynamoDB** → **Tables**.
- Click on the **Create table**



- Create a new table named **td_notes_archive**
- Using primary key is **user_id** and sort key is **timestamp**
- Keep all remaining setting as default
- Click on the create table

Create table

Table details Info

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.
 String

1 to 255 characters and case sensitive.

Sort key - optional
The sort key is the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.
 Number

1 to 255 characters and case sensitive.

- **td_notes_archive** table is created

Tables (5) info										
Find tables				Filter by tag Any tag key	Filter by tag value Any tag value		Last updated July 11, 2026, 14:10 (UTC+5:30)			
☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode
☐	Employees	Active	Empld (N)	-	0	0	Off	☆	Provisioned (5)	Provisioned (5)
☐	global_td-notes	Active	user_id (S)	timestamp (S)	0	0	Off	☆	On-demand	On-demand
☐	hotelRoomAvailabilityTable	Active	date (S)	-	0	0	Off	☆	On-demand	On-demand
☐	hotelRoomBookingTable	Active	bookingID (S)	-	0	0	Off	☆	On-demand	On-demand
☐	td_notes_archive	Active	user_id (S)	timestamp (N)	0	0	Off	☆	On-demand	On-demand

Step 2: Verify DynamoDB Streams

- Open the existing source table (**global_td-notes**).
- Navigate to **Exports and streams**
- Click on the Turn on (Right bottom)

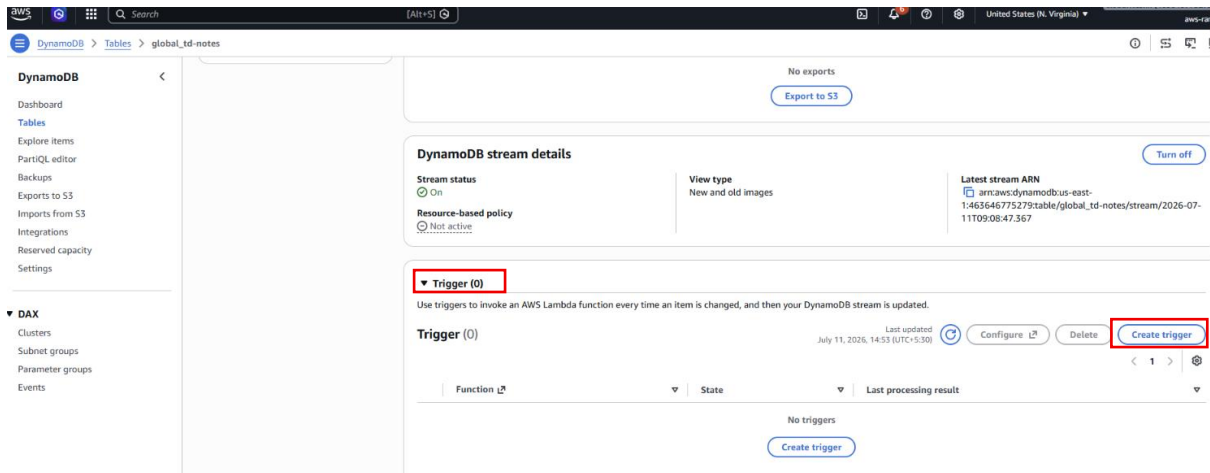
The screenshot shows the AWS DynamoDB console for the table 'global_td-notes'. The 'Exports and streams' tab is active. Under 'Exports to S3', it indicates 'No exports'. At the bottom, the 'DynamoDB stream details' section shows the stream status is 'Off', and a 'Turn on' button is highlighted with a red box.

- Verify that **DynamoDB Streams** is enabled with **New and old images**.
- Choose New and old images
- Click on the Turn on stream (Right bottom)

The screenshot shows the 'Turn on DynamoDB stream' dialog. In the 'View type' section, 'New and old images' is selected and highlighted with a red box. At the bottom right, the 'Turn on stream' button is highlighted with a red box.

Step 3: Create the Lambda Trigger

- Open the **Triggers** tab.
- Click **Create trigger**.



- Click on the **Create new Lambda function.**

Create a trigger

AWS Lambda function details
Use triggers to invoke an AWS Lambda function every time an item is changed, and then your DynamoDB stream is updated.

Lambda function
Choose an AWS Lambda function you want to trigger every time an item is changed.

Choose function **Create new**

Batch size
The number of records Lambda reads at once before invoking the function.

Between 1 and 10,000.

☒ **Turn on trigger**
Specify whether the trigger should be activated when you create it. You can turn it off later from the Lambda console.

Create trigger

- Choose **Use a Blueprint**
- Select Blueprint Name is **Process updates made to a DDB table**
- Name the function **Archive_notes**.
- Select **Use another role** in Execution role
- Choose the existing **AWSLambdaExecution-role**.

Create function

Choose one of the following options to create your function.

☐ Author from scratch
Start with a simple Hello World example.

☒ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ Container image
Select a container image to deploy for your function.

Basic information

Blueprint name
 nodejs22.x

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime
nodejs22.x

Architecture
x86_64

Execution role
To access other AWS services and resources your function needs an IAM role. Choose a role with the right permissions for your function.

☐ Create default role ☒ **Use another role**

Create new role

- Choose **global_td-notes** table
- Set the **Batch size** to **5**.
- Set the **Starting position** to **Trim Horizon**.
- Click on the **Create function**

DynamoDB trigger Remove

DynamoDB aws database event-source-mapping nosql polling

DynamoDB table Info
 Select or enter the ARN of a DynamoDB table. To select a stream in another AWS account, enter its DynamoDB stream Amazon Resource Name (ARN).

Event poller configuration

☒ **Activate trigger**
 Select to activate the trigger now. Keep unchecked to create the trigger in a deactivated state for testing (recommended).

☐ **Enable EventCount metrics**
 Track the number of events polled, filtered, invoked, and dropped by your event source mapping. CloudWatch charges apply.

Batch size Info
 The maximum number of records in each batch to send to the function.

Starting position Info
 The position in the stream to start reading from.

Batch window - optional
 The maximum amount of time to gather records before invoking the function, in seconds.

► **Additional settings**

In order to read from the DynamoDB trigger, your execution role must have proper permissions.

Cancel Create function

- Choose the Lambda function created just
- Set the **Batch size** to **5**.
- Leave the trigger **Disabled** during development.
- Click **Create trigger**.

Create a trigger

AWS Lambda function details
 Use triggers to invoke an AWS Lambda function every time an item is changed, and then your DynamoDB stream is updated.

Lambda function
 Choose an AWS Lambda function you want to trigger every time an item is changed.
 Create new

Batch size
 The number of records Lambda reads at once before invoking the function.

 Between 1 and 10,000.

☐ **Turn on trigger**
 Specify whether the trigger should be activated when you create it. You can turn it off later from the Lambda console.

Cancel Create trigger

Step 4: Edit the lambda Function

- Open the generated **AWS Lambda** function.
- Replace the default code with logic that processes **REMOVE** events from **Archive_notes**
- Instantiate the low-level **Amazon DynamoDB** SDK client instead of the Document Client.
- Loop through event.Records.
- Check whether eventName equals REMOVE.
- Read the expired item from record.dynamodb.OldImage.
- Write the item into the archive table using the **PutItem** operation.

```
'use strict';
```

```
const AWS = require("aws-sdk");
AWS.config.update({ region: 'us-west-1' });
```

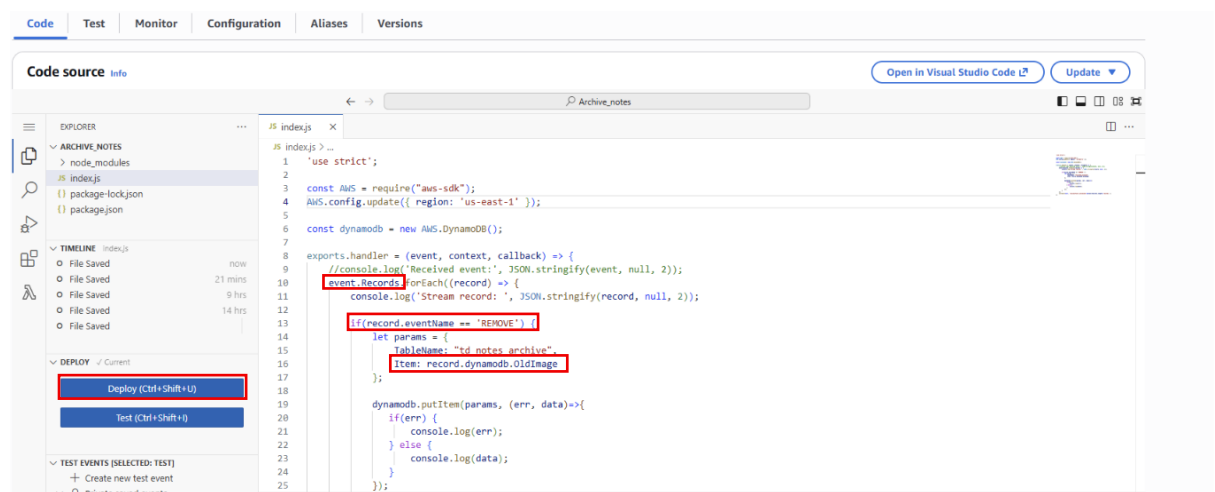
```
const dynamodb = new AWS.DynamoDB();
```

```
exports.handler = (event, context, callback) => {
  //console.log('Received event:', JSON.stringify(event, null, 2));
  event.Records.forEach((record) => {
    console.log('Stream record: ', JSON.stringify(record, null, 2));
```

```
if(record.eventName == 'REMOVE') {
  let params = {
    TableName: "td_notes_archive",
    Item: record.dynamodb.OldImage
  };

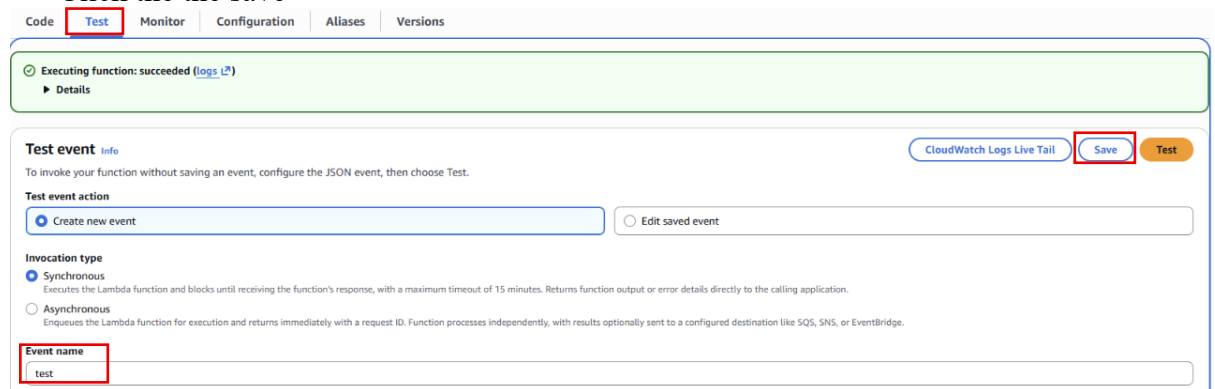
  dynamodb.putItem(params, (err, data)=>{
    if(err) {
      console.log(err);
    } else {
      console.log(data);
    }
  });
}
});
callback(null, `Successfully processed ${event.Records.length} records.`);
};
```

- Click on the **Deploy**



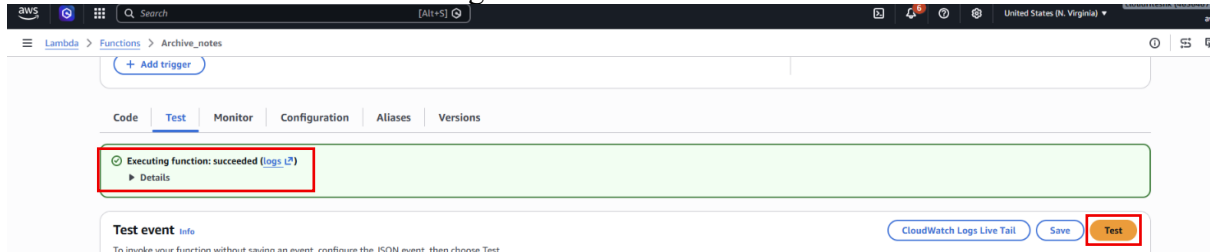
Step 5: Create Test event

- Go to the Test section.
- Create a Lambda **Test event** using a previously captured DynamoDB Streams event.
- Enter the Event name **test**
- Click the the save

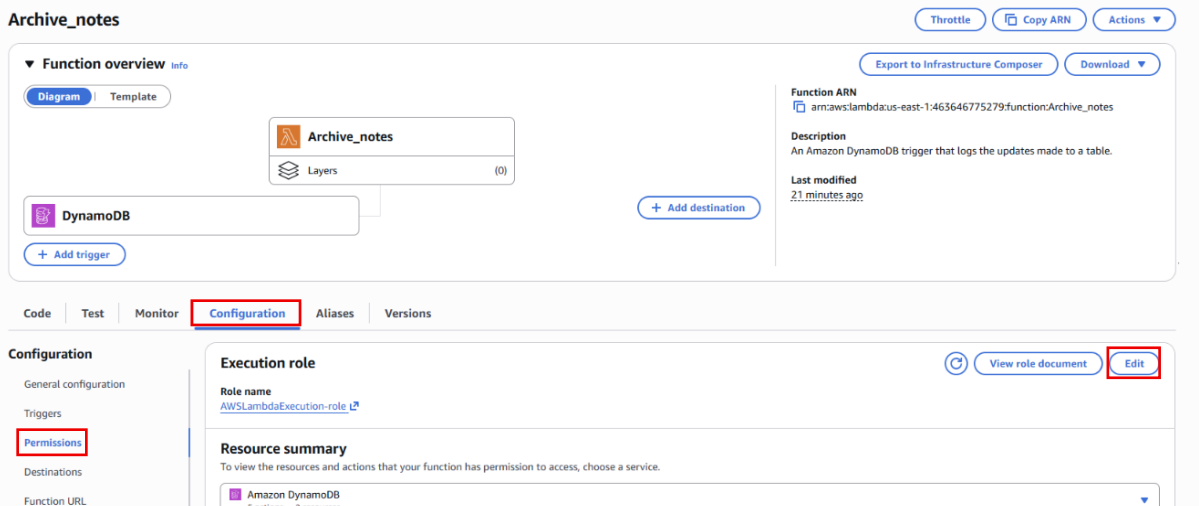


- Click **Test** to execute the function.
- Execute the function is succeed

- Click on the **Details** in Executing function succeeded

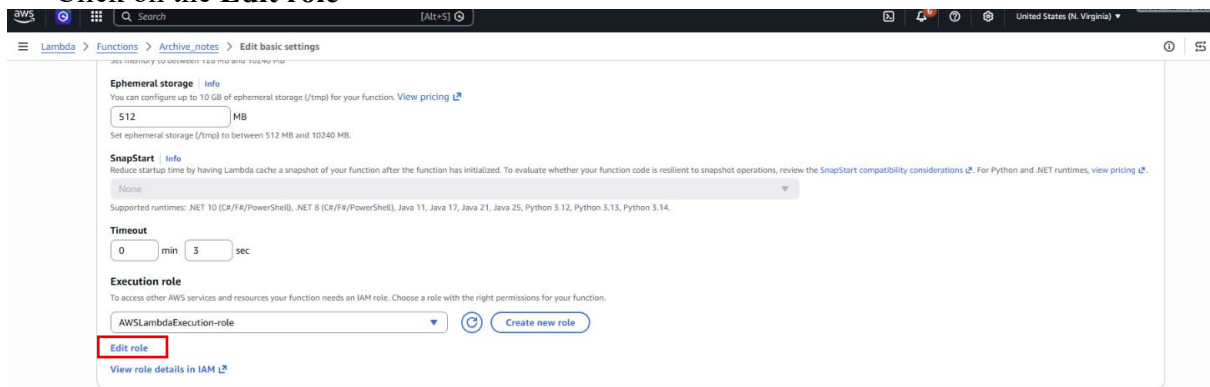


- Navigate to **Configuration**
- Go to **permission**
- Click on the **Edit**



Step 6: Update IAM permission

- If an **AccessDeniedException** occurs, navigate to **IAM** → open the Lambda execution role.
- Edit the attached IAM policy to allow **dynamodb**.
- Click on the **Edit role**

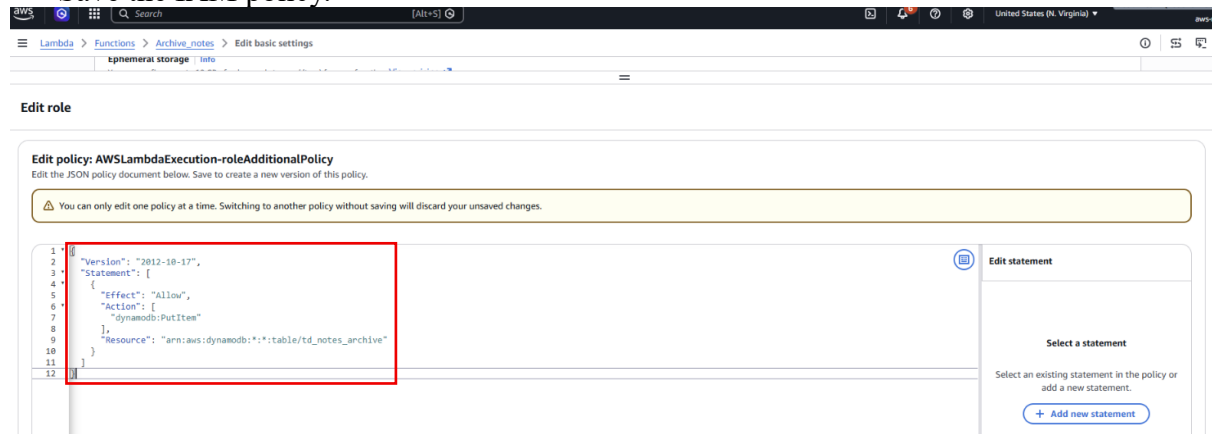


- Paste the Code

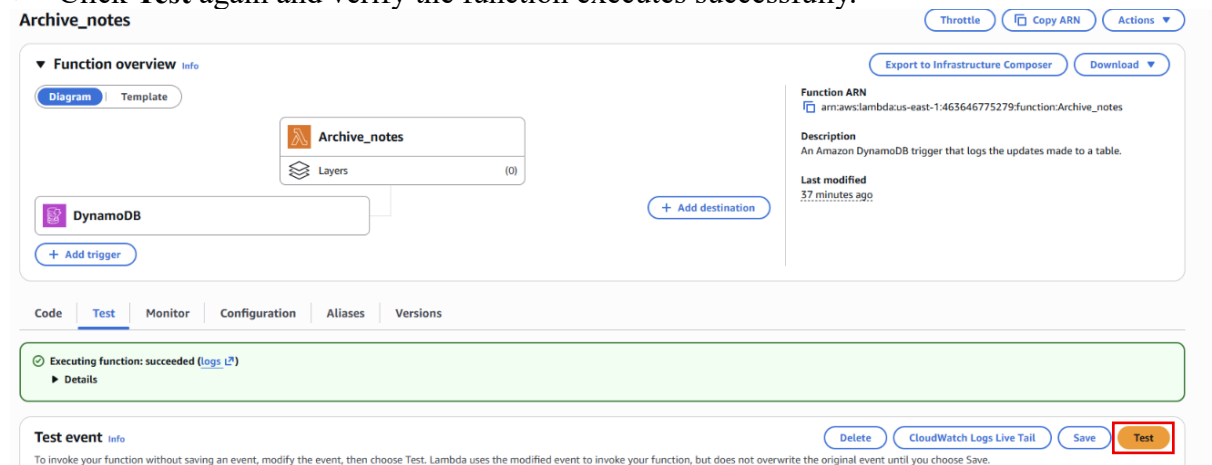
```
{ "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem"
      ],
      "Resource": "arn:aws:dynamodb:*:*:table/td_notes_archive"
    }
  ]
}
```

```
]
}
```

- Save the IAM policy.

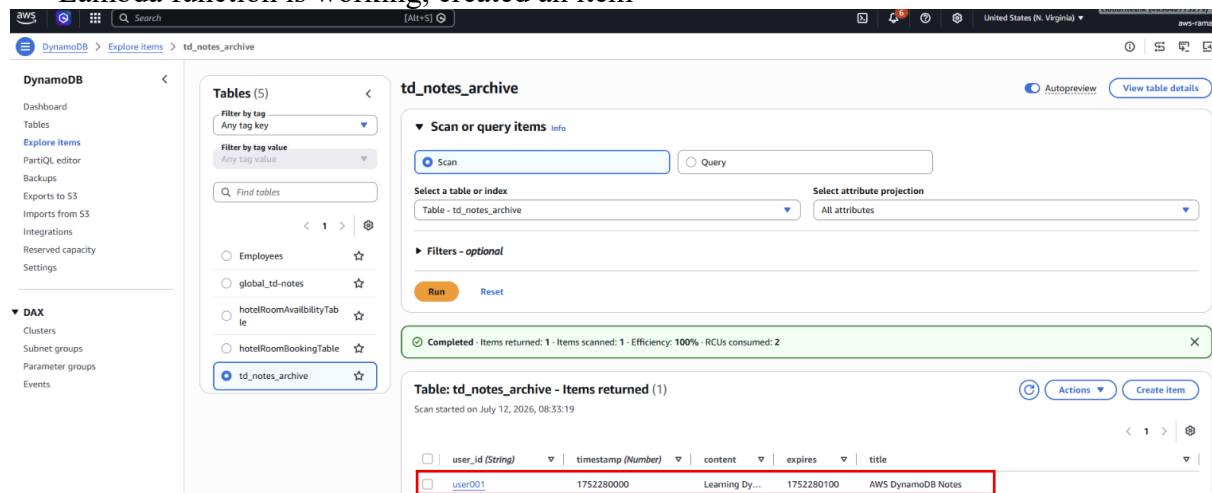


- Return to AWS Lambda.
- Click **Test** again and verify the function executes successfully.



Step 7: Verify the Archive Table

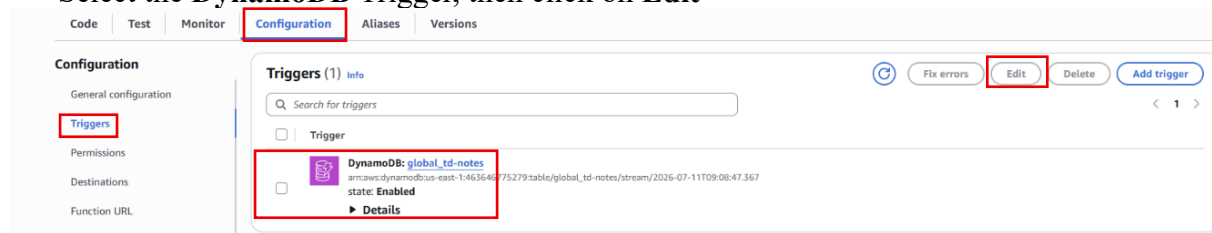
- Navigate to **Amazon DynamoDB** → **td__notes_archive**.
- Verify that the test records have been copied into the archive table.
- Lambda function is working, created an item



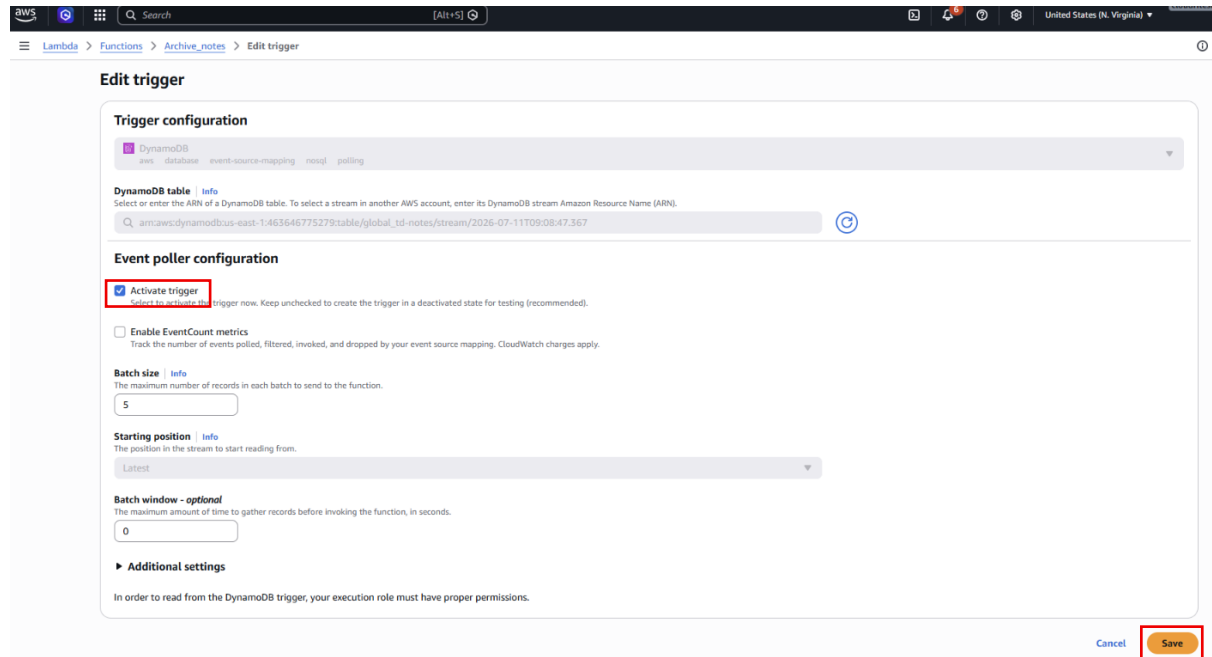
Step 8: Enable the Stream Trigger

- Return to the **AWS Lambda** function.
- Navigate to **Configuration**

- GO to the **Trigger**
- Select the **DynamoDB** Trigger, then click on **Edit**

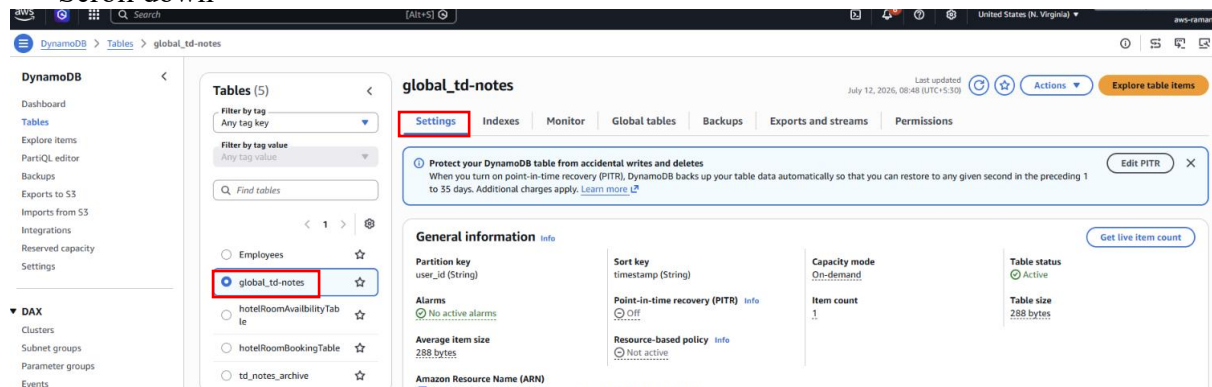


- Enable the **DynamoDB Stream Trigger**.
- Choose **Activate Trigger**
- Click on the **Save**

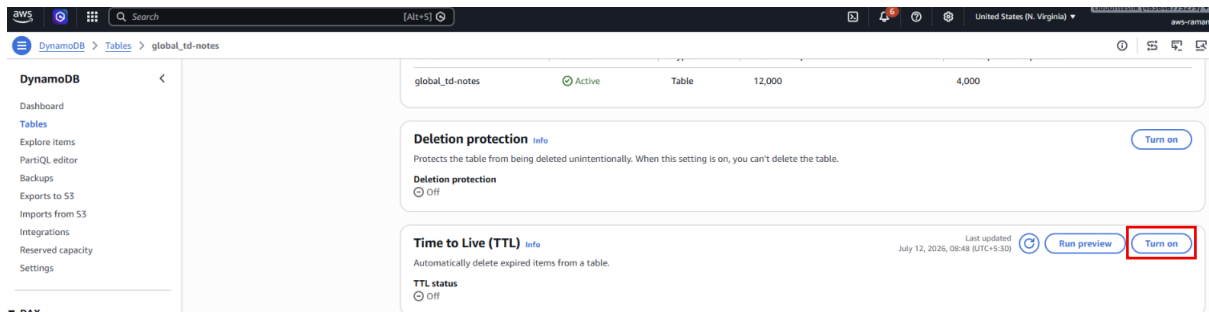


Step 9: Enable Time to live (TTL)

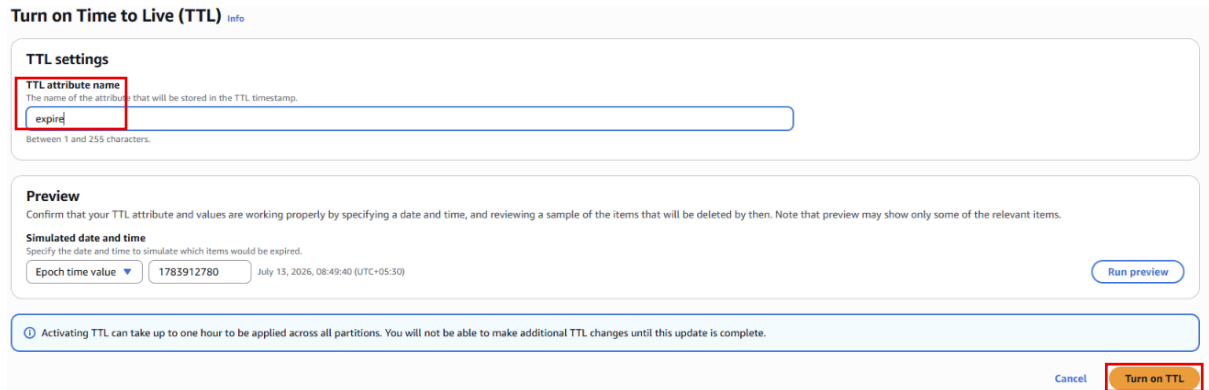
- Navigate to **Amazon DynamoDB** → **global_td-notes**.
- Go to the **Setting**
- Scroll down



- Open the **Time to Live (TTL)** settings.
- Click the **Turn on**



- Enable TTL using the **expires** attribute.
- Click on **Turn on TTL**



Step 10: Verify Automatic Archiving

- Wait for DynamoDB to remove expired items automatically (TTL operates on a best-effort basis and may take up to 48 hours after expiration).
- Open the archive table and verify that expired items are automatically copied.
- Navigate to **AWS Lambda** → **Monitoring** and verify successful invocations and logs in **Amazon CloudWatch Logs**.

